

day17笔记

一、Scheme数据模型

1、Scheme数据模型基础 --- 消息类型

```
from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
from langchain.schema import HumanMessage, AIMessage, SystemMessage
load_dotenv()

llm = ChatTongyi(model='qwen-plus')

# 表示的是用户输入消息
human_msg = HumanMessage(content='你好，请介绍一下你自己。')
print(f'消息类型: {type(human_msg).__name__}')
print(f'消息内容: {type(human_msg.content)}')

# 表示AI给用户的回复
ai_msg = AIMessage(content='你好，我是你的个人智能助理，小美。')
print(ai_msg.content)

# 表示的是需要给AI设定角色和行为规则（系统提示词）
system_msg = SystemMessage(content='你是一个专业的python编程高手，代码编写需要小白都能读懂，代码需要注释')
print(f'系统提示词: {system_msg.content}')
```

2、消息类型使用

```
from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
# 旧版本导入
# from langchain.schema import HumanMessage, AIMessage, SystemMessage
# 新版本导入（推荐）
from langchain_core.messages import HumanMessage, AIMessage, SystemMessage
load_dotenv()

llm = ChatTongyi(model='qwen3-max')

# 消息列表
# messages = [
#     SystemMessage(content='你是现代诗人，擅长书写男女之情相关的诗歌'),
#     HumanMessage(content='我叫丁致宇，失恋了，帮我写一手诗歌'),
# ]

messages = [
    SystemMessage(content='你是现代诗人，擅长书写男女之情相关的诗歌'),
    HumanMessage(content='我叫丁致宇，失恋了，给我写一首诗。'),
    AIMessage(content="""
        雨滴在窗上练习倒立行走
        像你名字里未拆封的晴空
    """)
]
```

我数着咖啡渍漫延的疆域
第七圈涟漪吞没了整座城池

衬衫第三颗纽扣突然松动
那是你去年冬天系上的结
如今它滚进沙发缝隙
变成一颗不肯融化的雪

手机屏保还停在樱花季
我们把誓言种成两株并蒂莲
可根系在暗处各自打结
开出背道而驰的春天

现在我要把心跳调成静音模式
任备忘录里的晚安集体叛逃
当所有未发送的早安沉入海底
新酿的月光会浮起另一艘船

"""),

HumanMessage(content='好诗，保持这个风格，再写一首关于丁致宇伤心的诗歌'),

]

输出结果

如果给模型的是一个消息列表，那么invoke这里传递需要使用关键字参数 input=消息列表

result = llm.invoke(input=messages)

print(result.content)

3、消息类型简化

```
from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
load_dotenv()
```

llm = ChatTongyi(model='qwen3-max')

消息列表

```
messages = [
    ('system', '你是现代诗人，擅长书写男女之情相关的诗歌'),
    ('human', '我叫丁致宇，失恋了，帮我写一手诗歌'),
    ('ai', ""
        雨滴在窗上练习倒立行走
        像你名字里未拆封的晴空
        我数着咖啡渍漫延的疆域
        第七圈涟漪吞没了整座城池
        ""
    ),
    ('human', '好诗，保持这个风格，再写一首关于丁致宇伤心的诗歌')
]
```

输出结果

如果给模型的是一个消息列表，那么invoke这里传递需要使用关键字参数 input=消息列表

result = llm.invoke(input=messages)

print(result.content)

二、stream流式输出

```
from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
load_dotenv()

llm = ChatTongyi(model='qwen3-max')

# 消息列表
messages = [
    ('system', '你是现代诗人，擅长书写男女之情相关的诗歌'),
    ('human', '我叫丁致字，失恋了，帮我写一手诗歌'),
    ('ai', """
        雨滴在窗上练习倒立行走
        像你名字里未拆封的晴空
        我数着咖啡渍漫延的疆域
        第七圈涟漪吞没了整座城池
    """),
    ('human', '好诗，保持这个风格，再写一首关于丁致字伤心的诗歌')
]

# 输出结果
# 如果给模型的是一个消息列表，那么invoke这里传递需要使用关键字参数 input=消息列表
result = llm.stream(input=messages)

for chunk in result:
    print(chunk.content, end='')
```

三、temperature温度

1、temperature可以控制AI输出内容的随机性和创造性，值范围0-2之间

2、特点

- 数值越低，回答越保守、确定，倾向于采用最稳妥、常见的词语回答你
- 数值越高，回答更开放、随机，也愿意使用一些不常用的词语进行组合来回答你

3、示例代码

```
from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
load_dotenv()

llm = ChatTongyi(model='qwen3-max', temperature=0)

result = llm.stream('请用一种非常独特的、非人类的视角，给‘爱情’下一个定义。')

for chunk in result:
    print(chunk.content, end='')

print('\n')
print('-' * 30)
```

```

llm2 = ChatTongyi(model='qwen3-max', temperature=1.2)

result2 = llm2.stream('请用一种非常独特的、非人类的视角，给‘爱情’下一个定义。')

for chunk in result2:
    print(chunk.content, end='')

```

四、给模型添加记忆

1、模型是没有记忆

```

from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
load_dotenv()

llm = ChatTongyi(model='qwen3-max')

result1 = llm.invoke('我喜欢吃苹果')
print(result1.content)

print('-' * 20)

result2 = llm.invoke('我喜欢吃什么水果? ')
print(result2.content)

```

哇，苹果真是个很棒的选择！🍏 它不仅清脆多汁、酸甜可口，还富含维生素和膳食纤维，对健康特别有益。你最喜欢哪种苹果呢？是红富士的甜美，还是青苹果的爽脆？或者其他特别钟爱的品种？🍏

我还不知道你喜欢吃什么水果呢！😄

不过，我可以帮你想想——你平时更喜欢甜的、酸的，还是清爽多汁的水果？比如：

- 喜欢甜的：芒果、香蕉、葡萄、荔枝
- 喜欢酸一点的：橙子、柚子、猕猴桃、青苹果
- 喜欢清爽多汁的：西瓜、哈密瓜、梨、草莓

如果你告诉我你的口味偏好，或者最近特别想吃的水果，我可以给你推荐一些哦！你最喜欢哪种水果？

2、添加记忆

```

from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
# 导入langchain记忆模块
# ConversationChain 和 ConversationBufferMemory 已经被废弃掉了
from langchain.chains import ConversationChain
from langchain.memory import ConversationBufferMemory
load_dotenv()

llm = ChatTongyi(model='qwen3-max')

# ConversationBufferMemory() 是专门用来给模型添加记忆的（之前需要一个聊天列表需要每次对话把聊天列表传递给模型）
# return_messages=True 作用是为了让咱们查看记忆相关的内容
emeory = ConversationBufferMemory(return_messages=True)

# ConversationChain 链，是langchain里面一个重要的模块，把模型和其他模块进行串联
chain = ConversationChain(llm=llm, memory=emeory)

```

```

result1 = chain.invoke('我喜欢吃苹果')
print(result1['response'])

print('-' * 20)

result2 = chain.invoke('我喜欢吃什么水果')
print(result2['response'])

print('-' * 20)

# emeory.load_memory_variables({}) 查看记忆相关内容
# history 历史记录
print(emeory.load_memory_variables({}))

```

```
chain = ConversationChain(llm=llm, memory=emeory)
```

哇，真棒！苹果不仅清脆多汁，还富含维生素C和膳食纤维，对身体特别好呢！你喜欢吃哪种苹果？是偏甜的红富士、酸甜适中的嘎啦果，还是特别脆的青苹果？我听说有些人还会把苹果切片蘸花生酱吃，或者做成

你刚刚提到“我喜欢吃苹果”，所以你现在最喜欢吃的水果是苹果呀！🍏

不过水果的世界可丰富啦——除了苹果，你还喜欢别的水果吗？比如香甜的芒果、多汁的西瓜、酸酸甜甜的草莓，还是热带风味的菠萝或百香果？每种水果都有独特的口感和营养，说不定还能发现新的心头好呢！😊

五、Embeddings Models 文本嵌入模型

- 1、作用：可以将字符串作为输入，返回一个浮点型的向量列表
- 2、在NLP中，把数据进行向量化
- 3、示例代码

```

from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
# 采用阿里的
from langchain_community.embeddings import DashScopeEmbeddings
load_dotenv()

embed_model = DashScopeEmbeddings(model='text-embedding-v1')

# 把文本向量化 --- 变成浮点数 --- 会计算两个词之间的向量距离（如果语义一样，那么它们的向量距离就比较近）
# 单次转换
# result1 = embed_model.embed_query('女孩脸红了，说："讨厌！"')
# result2 = embed_model.embed_query('女孩愤怒，说："讨厌！"')
#
# print(result1)
# print(result2)

# 批量转换
result3 = embed_model.embed_documents(['人工智能', '机器学习', '西红柿炒鸡蛋', '大模型是如何工作的', 'LLM的原理是什么'])
print(result3)

```

六、Prompt工程

1、什么是Prompt?

- Prompt是提示词，是用户和语言模型沟通的方式

2、为什么要学习Prompt?

场景

- 给我讲过笑话
- 使用python给我写个小游戏
- 帮我写个企业官网

问题

- 给我讲过笑话，没有说笑话类型，什么场合下（时间、地点、受众），给出答案肯定不是你想要的
- 使用python给我写个小游戏，什么类型的游戏，游戏场景、游戏玩家年龄、游戏关卡...
- 帮我写个企业官网，没有告诉模型你的企业是做什么的，官网模块有那些，需不需要做官网后台...

提示词例子：

- 我要吃饭
- 我要吃一份西红柿炒蛋，不要放葱花，米饭少点儿，使用保温袋来送

3、例如：

让AI给咱们做事情（坏的提示词）

```
prompt = [  
    '帮我写代码',  
    '帮我写点儿东西',  
    '这是什么问题',  
    '这个怎么写'  
]  
# 1、角色设置  
# 2、具体任务描述  
# 3、要求是什么  
# 4、以什么格式输出
```

好的提示词

```
prompt1 = """  
角色：你是一位Python后端工程师，擅长FastAPI框架  
  
任务：帮我写一个用户注册API接口  
  
要求：  
1. 使用FastAPI  
2. 包含用户名、邮箱、密码（密码需加密存储）  
3. 返回JWT Token  
  
输出格式：完整可运行的代码，包含必要的注释
```

```
"""

prompt2 = """
角色：你是一位科技博主，擅长写通俗易懂的科普文章

任务：写一篇关于人工智能的科普文章

要求：
1. 面向零基础读者
2. 包含3个生活化例子
3. 控制在800字以内

输出格式：Markdown格式，带标题和小标题
"""
```

七、通用（基础）提示词模版

1、基础提示词模版三种使用方式

```
from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
# 导入提示词模版
from langchain_core.prompts import PromptTemplate

load_dotenv()

llm = ChatTongyi(model='qwen3-max')

# name = '张三'
# age = 18
# prompt = f'你好: {name}, 年龄: {age}'
# prompt = '你好: ' + name + '年龄: ' + str(age)
# print(prompt)

# 第一种写法
# 创建模版
# template = PromptTemplate(
#     input_variables=['name', 'age'], # 变量占位
#     template='你好: {name}, 年龄: {age}'
# )
# 填充模版
# prompt = template.format(name='张三', age=18)

# 调用模型
# result = llm.invoke(prompt)
# print(result.content)

# 第二种写法（推荐写法）
# template = PromptTemplate.from_template('你好: {name}, 年龄: {age}')
# prompt = template.format(name='张三', age=18)
# print(prompt)
```

```
# 第三种写法
# template = PromptTemplate.from_template('你是一个专业的律师，请你回答我提出的法律问题，并给出法律条文依据，我的问题是: {question}')
```

注意点: template.invoke() 参数必须传递字典

```
# prompt = template.invoke({'question': '婚姻法是哪一年颁布的? '})
# result = llm.invoke(prompt)
# print(result.content)
```

2、带有格式的提示词模版

```
format_template = PromptTemplate.from_template("""
    任务: {task}
    时间: {time}
    地点: {location}

    根据上面的格式（信息）生成一段话
    """)

prompt = format_template.format(task='开会', time='明天下午3点钟', location='二楼会议室')
print(prompt)
```

3、提示词模版代码审查案例

```
from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
from langchain_core.prompts import PromptTemplate

load_dotenv()

llm = ChatTongyi(model='qwen3-max')

code_template = PromptTemplate.from_template("""
# 角色: 资深代码审查专家
你精通 Python 安全规范和 PEP8 标准。

# 任务: 审查以下代码
{code}

# 审查维度（请打分 1-10）
1. 安全性（是否有注入风险）：
2. 规范性（命名是否符合 PEP8）：
3. 逻辑漏洞：

# 输出格式
请使用 JSON 格式输出，包含 "score", "issues" 字段。
""")

code = """
def calc(a, b):
    return a / b
"""

prompt = code_template.format(code=code)
```

```
result = llm.invoke(prompt)

print(result.content)
```

八、聊天型提示词模版

- 1、聊天型提示词模版，有消息类型的存在（system、human、ai）
- 2、示例代码

```
from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
from langchain_core.prompts import PromptTemplate, ChatPromptTemplate

load_dotenv()

llm = ChatTongyi(model='qwen3-max')

chat_template = ChatPromptTemplate.from_messages([
    ('system', '你是一个资深的python工程师，请认真回答我提出的问题，并确保回答没有错误'),
    ('human', '写一个python程序，关于{question}')]
)

chat_prompt = chat_template.invoke({'question': '冒泡排序'})

result = llm.invoke(chat_prompt)
print(result.content)
```

- 3、带有历史记录的提示词模版

```
from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

load_dotenv()

llm = ChatTongyi(model='qwen3-max')

# 电商客服相关聊天型提示词模版
# system 设定人设（这部分是不会改变）
# MessagesPlaceholder() 这个是历史记录的占位（坑 --- 等着内容填）
# input 表示的是用户输入的新问题
chat_template = ChatPromptTemplate.from_messages([
    ("system", """
    你是优品商城的智能客服 '小E'。
    1. 退换货政策：7天无理由。
    2. 如果不知道答案，礼貌回复并建议转人工。
    3. 保持亲切的语气。
    """),
    MessagesPlaceholder(variable_name='chat_history'),
    ('human', '{input}')
```

```

])

# 用户输入
current_input = '对了，那运费谁承担呢？'

# 历史记录
chat_history_data = [
    ("human", "你好，我想买个吹风机。"),
    ("ai", "亲，您好！我们有新款负离子吹风机。"),
    ("human", "如果不喜欢可以退吗？"),
    ("ai", "可以的亲，支持7天无理由退换货。")
]

# 调用提示词模版，并填充内容
final_messages = chat_template.format_messages(
    chat_history = chat_history_data,
    input=current_input
)

# 把聊天型提示词模版输出
# for msg in final_messages:
#     print(f'消息类型: {msg.type}')
#     print(f'内容: {msg.content}')

result = llm.invoke(final_messages)
print(result.content)

```

九、小样本提示词模版

1、零样本提示 (Zero shot prompt)

```

from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder, PromptTemplate
from sympy.physics.units import temperature

load_dotenv()

llm = ChatTongyi(model='qwen3-max')

zero_shot_template = PromptTemplate.from_template(
    '请判断以下评论的情感是正面、负面还是中立：\n评论: {comment}\n情感: '
)

prompt = zero_shot_template.format(comment='手机收到了，使用了几天才黑屏了')

result = llm.invoke(prompt)
print(result.content)

```

2、小样本提示 (few shot prompt)

```

from sys import prefix

```

```

from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder, PromptTemplate,
FewShotPromptTemplate

load_dotenv()

llm = ChatTongyi(model='qwen3-max')

# 准备的示例
examples = [
    {"comment": "这绝对是今年我用过最棒的耳机，音质清晰，降噪无敌！", "sentiment": "正面"},
    {"comment": "太差劲了，才用了一周就坏了，客服也联系不上。", "sentiment": "负面"},
    {"comment": "包裹已经收到了，外包装有点磨损。", "sentiment": "中立"}
]

examples_template = PromptTemplate.from_template(
    '评论: {comment}\n情感: {sentiment}'
)

# examples 小样本（例子）
# example_prompt 最终以我模版里面的格式输出
# prefix 系统指令（system）
# suffix 用户输入（用户提示语）

# 小样本提示的作用，主要是为了通过例子来引导AI给出标准的回复
few_shot_prompt = FewShotPromptTemplate(
    examples=examples,
    example_prompt=examples_template,
    prefix='请根据以下示例，判断评论的情感（正面、负面、中立）',
    suffix='评论: {comment}\n情感',
    input_variables=['comment'],
)

final_prompt = few_shot_prompt.format(comment='手机收到了，系统非常流畅')

result = llm.invoke(final_prompt)
print(result.content)

```